

Individual Technical Report

SpiderPi Mobile Control and AI Mission Commander

Author: Magne Dina Neves

Student ID: Dc328753

Role: Backend Implementation and System Integration

Course	ECEN3025 IOT Workshop
University	University of Macau
Project Type	Group Project (2 members)
Teammate	Carlos Jorge DC328735
Date	May 1, 2026

Abstract

This report documents my implementation contribution to the SpiderPi project. I developed the Python control backend (`phone_control_server.py`), integrated web-based teleoperation, built an autonomous mission executor, added camera proxy delivery for browser compatibility, and implemented an AI planning endpoint with deterministic fallback. My implementation goal was to preserve SpiderPi native movement reliability while introducing practical autonomy with safe interruption and recovery behavior. The final system supports manual control, autonomous missions, mission status telemetry, and resilient runtime behavior in lab/demo conditions.

1 Project Objective and Engineering Strategy

The baseline SpiderPi software provided low-level robot capability through:

- `SpiderPi.py` (main runtime),
- `RPCServer.py` (JSON-RPC motion APIs),
- `MjpgServer.py` (camera stream server).

The target was to deliver:

1. a phone browser dashboard for control,
2. autonomous mission execution with safe stop,
3. camera visualization in the same dashboard,
4. robust behavior in unstable network/demo environments.

I followed a **reuse-first strategy**: avoid rewriting low-level kinematics; instead, add a robust orchestration layer above existing RPC methods.

2 Implemented Architecture

The deployed architecture has three layers:

1. **UI Layer:** mobile web dashboard (`web/index.html`, `web/app.js`, `web/styles.css`).
2. **Control Layer:** Python bridge and mission engine (`phone_control_server.py`).
3. **Execution Layer:** native SpiderPi RPC motion and action-group runtime.

2.1 Data Flow

1. Phone UI sends command/mission request to Python backend.
2. Backend validates request and maps it to SpiderPi JSON-RPC.
3. Robot executes movement/action/mode transition.
4. Backend returns status/logs; UI updates telemetry and camera panel.

3 Backend Implementation Details

3.1 Core Libraries and Justification

I selected Python standard library components to reduce installation risk on 32-bit Raspberry Pi:

- `http.server`: REST-style API + static file serving.
- `urllib.request`: internal RPC forwarding.
- `threading`: background mission and keepalive workers.

- json, time, re, logging: payload handling, parsing, observability.

Justification: lower dependency complexity, predictable startup, better portability during demo constraints.

3.2 API Endpoints Implemented

Endpoint	Method	Purpose
/api/health	GET	Service liveness, last command, mission summary fields.
/api/command	POST	Manual command execution (move, stop, stand, action).
/api/ensure_mode	GET	Reinitialize SpiderPi remote mode (LoadFunc(1), StartFunc()).
/api/camera.jpg	GET	Camera snapshot proxy from local robot camera endpoint.
/api/mission/plan	POST	Create mission plan from text prompt (AI or heuristic fallback).
/api/mission/start	POST	Start validated mission plan.
/api/mission/stop	POST	Interrupt active mission safely.
/api/mission/status	GET	Poll mission runtime state and logs.

3.3 Command Mapping to Robot Motion

I mapped manual intents to existing action groups and RPC functions to minimize risk:

- forward/back/left/right → RunAction(ID,times)
- stop → RunAction("0",1)
- stand → Stand(160,2,500)
- wave/dance/kick → RunAction with known IDs

Listing 1: Manual command routing excerpt

```

if command in ("forward", "back", "left", "right"):
    action_id = {"forward": "1", "back": "2", "left": "3", "right": "4"}[command]
    times = {"low": 1, "medium": 2, "high": 3}.get(speed_label, 1)
    return self._rpc_ok("RunAction", [action_id, times], command)

if command == "stop":
    return self._rpc_ok("RunAction", ["0", 1], command)

```

4 Mission Engine (Implemented)

I implemented an autonomous mission runtime with:

- plan validation,

- sequential step execution,
- operator interrupt,
- mission logs and state reporting,
- safe post-mission recovery.

4.1 Supported Mission Step Types

Step Type	Execution Semantics
autoavoid(duration)	Load avoidance mode, keep alive with heartbeat, stop and recover.
move(direction,speed)	Map direction to action-group movement and execute bounded repeats.
action(name,times)	Execute expressive action (wave, dance, kick, stand).
wait(duration)	Controlled pause for sequencing and choreography.
stand	Return to standard stable posture.

Listing 2: Validation bounds excerpt

```

if step_type == "autoavoid":
    duration = int(step.get("duration", 20))
    duration = max(5, min(120, duration))
elif step_type == "move":
    direction = step.get("direction", "forward").lower()
    if direction not in ("forward", "back", "left", "right"):
        raise ValueError("invalid move direction")
    repeats = max(1, min(8, int(step.get("repeats", 1))))

```

5 AI Planning Integration

I added an AI planning endpoint:

- cloud planner path (when API key is available),
- deterministic heuristic parser fallback (always available).

5.1 Why This Design

1. Keep advanced planning capability for innovation.
2. Preserve full operability without internet/cloud.
3. Reduce demo failure risk due to external service outage.

6 Camera Integration and UI Reliability

Direct browser MJPEG access was not consistently reliable across devices and network policies. I implemented a backend camera proxy:

- UI requests `/api/camera.jpg`
- backend fetches local snapshot from robot camera service
- UI updates frame in a timed refresh loop

This solved compatibility issues in restrictive network/browser conditions.

7 Safety and Reliability Mechanisms Added

Mechanism	Purpose and Effect
Heartbeat keepalive thread	Prevent automatic mode timeout during long mission runs.
Command throttling	Reduce unstable burst traffic from rapid touch events.
Mission/manual arbitration	Reject non-stop manual commands while mission is active.
Emergency stop path	Allow immediate interruption priority over mission sequence.
Safe-state recovery	On mission end/error, force stop, unload function, return to manual mode.
Runtime logs/status endpoint	Improve debugging speed and real-time observability.

8 Testing and Validation Summary

8.1 Functional Validation

- Manual control: forward/left/right/back/stop/stand/action.
- Mission flow: plan → start → running → stop/recover.
- Camera: repeated frame fetch under motion load.
- Mode switching and return to manual state.

8.2 Network Validation

- Shared lab Wi-Fi mode.
- Robot AP fallback mode.
- Mixed phone/hotspot scenarios.

9 Representative Mission Example

Listing 3: Mission JSON example

```
{
  "name": "lab_showcase",
  "steps": [
    {"type": "autoavoid", "duration": 30},
    {"type": "action", "name": "wave", "times": 1},
    {"type": "stand"}
  ]
}
```

10 Conclusion

My contribution delivered the full implementation bridge from mobile UI to native SpiderPi execution, extended with supervised autonomy and robust runtime protection. The final system is not only a remote-control interface; it is a structured autonomous control platform with safe interruption, deterministic recovery, and deployment-ready behavior for real lab demonstrations.

Comparing with the midterm proposal a lot of things changed, highlighting the system implementation as it was firstly designed for full autonomous medical care, although our current project can still do tasks if needed it will need upgrades in terms of sensors for enabling automatic and autonomous control. We did have success implementing the hand gesture and voice command (through mobile web application features) and that cleared the pre-built limitations in terms of microphones and speakers.

In appendix to this report are all the files, ML models and relevant materials for proper evaluation